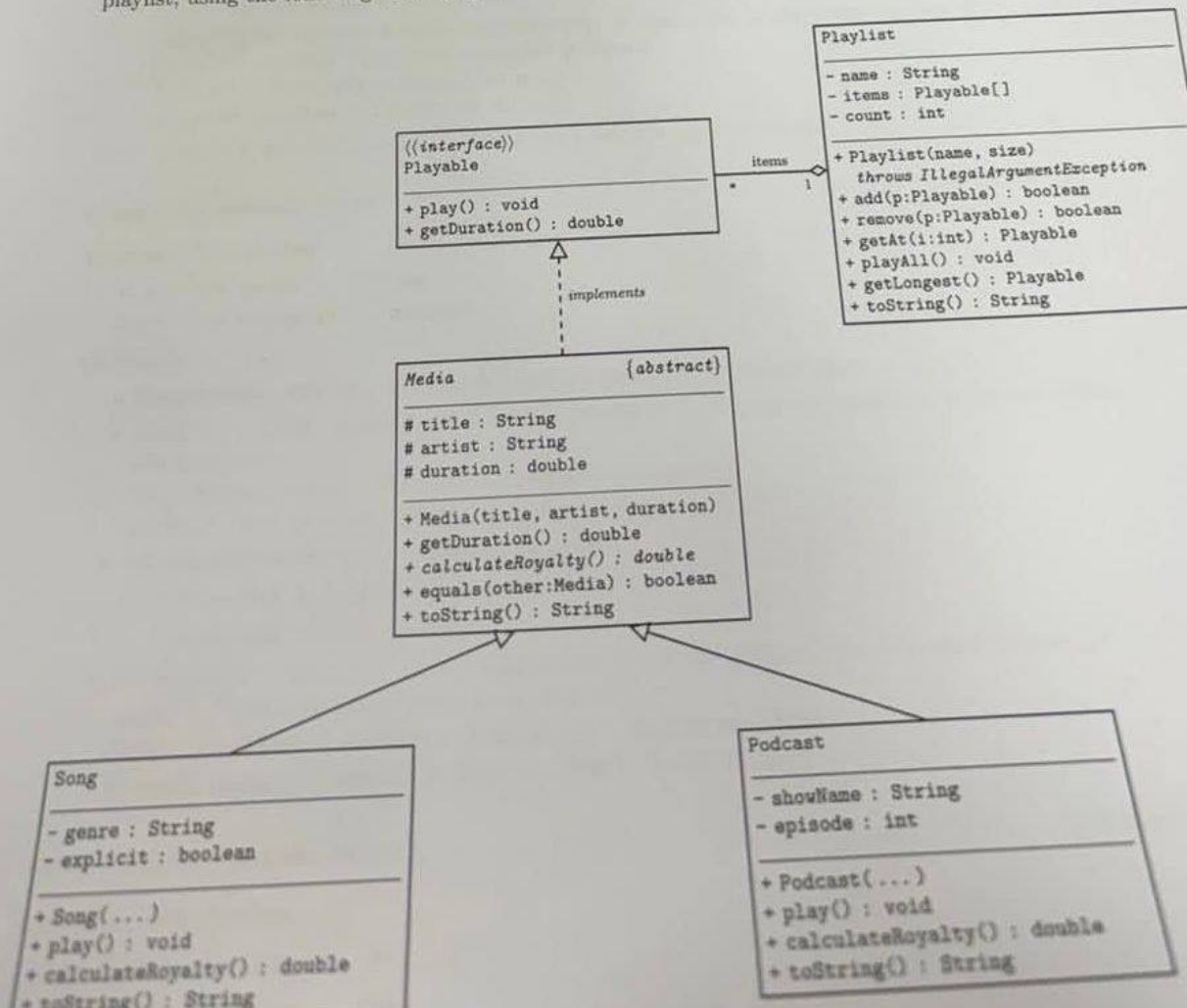
Instructions

- The use of any Java Collection or external library is **not allowed**.
- Open a new workspace before starting in your IDE.
- Write your name and ID at the top of every `.java` file as a comment.
- Submit all `.java` files to LMS. Any file not submitted receives **0**.
- Duration: **1 hour 50 minutes**. Total marks: **10**.

Scenario

We want to write a program that manages playable media (songs and podcasts) inside a streaming playlist, using the following UML diagram:



Specification

Interface `Playable`

Methods (abstract): [0.5 marks]

- `play()` : `void` : plays the item.
- `getDuration()` : `double` : returns the duration of the item in minutes.

Abstract Class `Media` (*implements* `Playable`) [1.5 marks]

Instance Attributes:

- `protected title` : `String`
- `protected artist` : `String`
- `protected duration` : `double`

Methods:

- `Media(title, artist, duration)` : constructor.
- `getDuration()` : `double` : returns `duration`.
- `calculateRoyalty()` : `double` : **abstract**.
- `equals(other : Media)` : `boolean` : two media items are equal if they have the same `title` **and** the same `artist`.
- `toString()` : returns a string representation of the media in the following format (`< . . . >` marks an attribute value, everything else is literal):
  `<title> - <artist> (<duration> min)`
  Example: `Imagine - Lennon (3.03 min)`

Note: `play()` is not implemented in `Media`; each subclass provides its own.

Class `Song` ***extends*** `Media` [1.25 marks]

Instance Attributes:

- `private genre` : `String`
- `private explicit` : `boolean`

Methods:

- `Song(title, artist, duration, genre, explicit)` : constructor.
- `play()` : `void` : prints in the following format (`< . . . >` marks an attribute value, everything else is literal):
  `Now playing song: <title> - <artist> [<genre>]`
  Example: `Now playing song: Imagine - Lennon [Pop]`
- `calculateRoyalty()` : `double` : returns:
  - `duration * 0.10` if the song is explicit
  - `duration * 0.05` otherwise
- `toString()` : returns a string representation of the song in the following format (`< . . . >` marks an attribute value, everything else is literal):
  Song: `<title> - <artist> [<genre>] (<duration> min)`
  Example: Song: `Imagine - Lennon [Pop] (3.03 min)`

lass *Podcast* ***extends*** `Media` [1.0 mark

stance Attributes:

- *private showName* : `String`
- *private episode* : `int`

- `Podcast(title, artist, duration, showName, episode)` : constructor.
- `play()` : `void` : prints in the following format (`<...>` marks an attribute value, everything else is literal):
  `Streaming podcast: <title> -- <showName> Ep#<episode>`
  Example: `Streaming podcast: Tech -- TED Ep#12`
- `calculateRoyalty()` : `double` : returns `duration * 0.02`.
- `toString()` : returns a string representation of the podcast in the following format (`<...>` marks an attribute value, everything else is literal):
  `Podcast: <title> - <artist> | <showName> Ep#<episode> (<duration> min)`
  Example: `Podcast: Tech - Doe | TED Ep#12 (42.30 min)`

Class `Playlist` **[3.75 marks]**

Instance Attributes:

- `private name : String`
- `private items : Playable[]`
- `private count : int`

Methods:

- `Playlist(name, size)` : constructor.
  If `size <= 0`, the constructor must explicitly throw:
  `throw new IllegalArgumentException("Invalid size: " + size);`
  (`IllegalArgumentException` is a Java built-in runtime exception.)
- `add(p : Playable) : boolean` : adds `p` to the first available space if there is space and `p` is not already in the playlist (use `equals` for comparison). Returns `true` if it succeeds, `false` otherwise.
- `remove(p : Playable) : boolean` : removes `p` from the playlist if it is there by replacing it with the last item, then decrements `count`. Returns `true` if it succeeds, `false` otherwise.
- `getAt(i : int) : Playable` : returns the item at index `i` (index starts from 0), or `null` if `i` is out of range.
- `playAll() : void` : calls `play()` on every item in the playlist (polymorphism through the `Playable` interface).
- `getLongest() : Playable` : **recursively** returns the item with the longest duration in the playlist, or `null` if the playlist is empty.
- `toString()` : returns a string representation of the playlist in the following format:

```
name Playlist (count):
item1
item2
...
```

`getLongest()` must be implemented recursively. An iterative solution will receive 0 marks.